

TECHNICAL DESIGN DOCUMENT - RHYTHM

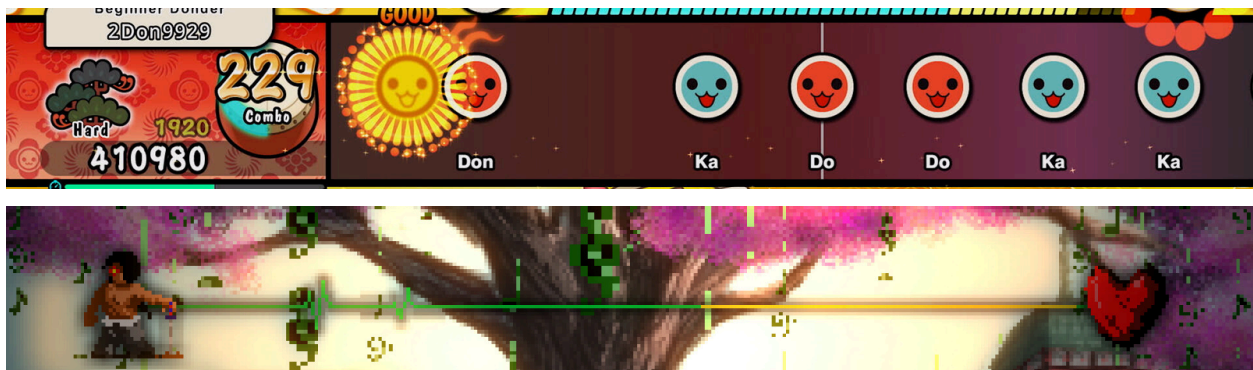
DESCRIPTION:

This Document is intended as a way to visualize and to plan the work required in order to create the rhythm minigames in our currently untitled Game Production Final, It is intended as a way to describe and explain the systems in a way where all is understood by all members of the team. If there is any confusion please refer to Tino for any sort of explanation or instruction.

REFERENCES:

This section is dedicated to understanding the rhythm genre, here are a few examples of the different types of rhythm games.

ONE-TRACK



One track rhythm games are the simplest style of rhythm game and require the player to hit one or two buttons as they approach from one end of the screen to the next. We will be creating one track rhythm sections for the sake of brevity in this project.

MULTI-TRACK

A multi track game would not be impossible for us to make, they're generally the same thing coding wise except for the Charting (The actual rhythms that the player is expected to perform i.e the level of the rhythm game). Charting all these levels will take significant amounts of time and having more tracks WILL make things harder. This HEAVILY depends on whether or not other people will be comfortable learning charting.



RHYTHM GAME SETUP:

Upon entering a level, the game creates a few objects the player needs to collect before progressing with the level, with the objects being placed randomly in a specific range. When interacting with the objects, the game runs certain scripts that begin the rhythm mini game to obtain the item.

HOW TO MAKE IT WORK:

There is one main hit section that has a note come towards it, you then click a button when the note is over the hit section and its proximity to the hit section's center decides how good of a score you receive.



Your Hit Section (The area that the note is heading towards) Has multiple colliders inside of it each one varying in size (Referred to as zone Colliders). The note has a very very tiny collider (referred to as NoteCollider) and when a button on the controller is pushed the three colliders activate and check if the note is on top, depending on which Zone Colliders the NoteCollider is touching at the time you are then awarded a certain amount of something (Points/Gratification[GREAT, Good, okay])

The game needs to have a configurable metronome (tells the computer when and where the beats are and is adjustable to the speed of the song) and a way to make the charts in unity. It also has to be understandable by people other than me (tino).

Charting!

Charting is the concept of deciding when you have to hit notes to the music, deciding which rhythms the player will perform and also the difficulty of the rhythm section. If I am the only person charting everything, this will not go well! So I will need help doing this or for this to be my singular task (preferably not). There are a few ways to do this such as:

Writing in the code:

We create a game object that will always move towards the goal in a straight line, we then instantiate it at different intervals to the beat of the song, this is hard cause you don't just want them to spawn at the beat of the song but also land to the beat of the song but easier to sync up than off screen.

Off-Screen:

Each chart is a series of notes that all move to the goal in a straight line but instead of spawning them in, they are all off screen for as long as we need them to be. We load them all at once and have them arrive on time.

MIDI:

This is the really really cool option that's accessible to us due to the fact that I am here. MIDI is a standardized form of writing sheet music. In order to write MIDI files you have to understand music theory and how it all works, but since MIDI files are digital it turns this process into something very shockingly easy. I had been under the idea that this would be insanely difficult but as of the time I'm writing this I found a tutorial that seems great, and is only 40 minutes long. CAVEAT, I'm the only one here who could write and understand sheet music. That means I would have to be involved in every single song in the game. (I'll be fine lol)

TUTORIALS:

These are youtube playlists/Videos of tutorials on how all of this works and is programmed

▶ **The Key Components of a Rhythm Game** Explains it all from a non technical standpoint but from a generalist game design standpoint and isn't specific to unity

▶ **How To Make a Rhythm Game #1 - Hitting Notes** This is the first of a 5 part tutorial from 2014 regarding all of the Unity programming that goes into a rhythm game, the end result that he makes is hideous and the programming is shaky and not fully relevant anymore BUT a strong resource regardless

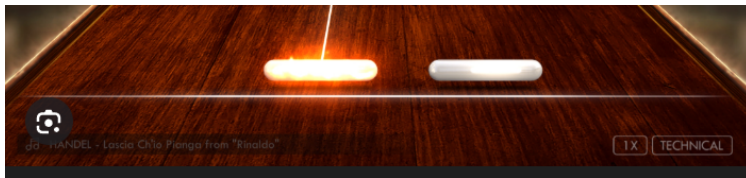
▶ **How to make a Rhythm Game In Unity (using MIDI)** This one is better than the previous, BUT is wayyy more complex and requires MIDI. Charting would be super super easy but the overall script is gonna be way harder for me to tweak. (*leaning heavily towards this one*)

▶ **SYNC YOUR UNITY GAME TO THE BEAT** This is a system of syncing up an entire world to do things in relation to the beat of music, we could use this to have lights pulsate to the beat and other cool effects. This code is also basically what we would need for a configurable metronome

ART ASSETS REQUIRED:

Landing Area/Zone :

This is the Hit Section that the notes will eventually land on TOP of (must be visually distinct color wise). this can be super simple and be a singular white line, or something more complicated as long as it fits the general shape of the notes approaching it



Notes Visual:

This is the actual note itself that will reach the main line, must be the same general shape, (squares falling onto other squares or circles falling onto other circles) we must also have alternates for when you get the note correct so that they turn white or have some other effect applied to make it feel good



Congratulatory effect:

This is typically the graphic text and vfxs that will play depending on how good of a job you do, for our purposes we need something that can represent GREAT, good, and bad/miss minimum



UI ASSETS:

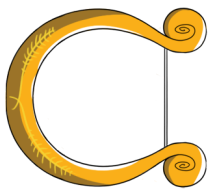
Blacksmith Head - Asset used during boss battles to represent the player character



Diamonds - Assets representing the notes the player has to hit to beat the rhythm mini game.



Lyre



Strings - Asset representing where notes will appear from for the player to interact with.

